

المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⵉⵎⵙⵙⵓⵔⵜ ⵜⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵏⴰⵡⵉⵏ ⵜⵜⴰⵡⵓⵏⵉⵏ ⵜⵉⵜⵡⴰⵏ
Ecole Nationale des Sciences Appliquées de Tétouan

Application des Méthodes Numériques à la Résolution de Problèmes Algébriques à l'aide de Python

École nationale des sciences appliquées de Tétouan

Filière : Science des Données, Big Data et IA

Département : Intelligence Artificielle et Digitalisation (IAD)

Encadré par :

M. Adel Louly

Réalisé par :

Yassir Adila

Apogée : 22048197
CNE : P110072435

Date de soumission : 23 / 05 / 2025

Contents

1 Introduction **3**

2 Exercice 1 : Résolution de Systèmes Linéaires **5**
2.1 Méthodes Directes : Décomposition LU 5
2.2 Méthode de Cholesky 8
2.3 Méthodes Itératives : Jacobi et Gauss-Seidel 10

3 Exercice 2 : Décompositions SVD et QR **14**
3.1 Décomposition SVD 14
3.2 Décomposition QR 17

4 Exercice 3 : Application à l'Analyse de Données **20**
4.1 Analyse en Composantes Principales (ACP) 20
4.2 Compression d'Image 24

5 Questions **34**
5.1 Comparaison des performances (temps, précision) des méthodes directes et itératives pour résoudre $Ax = b$ 34
5.2 Impact de la réduction de dimension via SVD sur l'analyse de données . . . 35
5.3 Avantages et limites de l'ACP pour l'analyse de données 36

6 Résumé et Conclusion **37**

1 Introduction

Ce projet explore les techniques fondamentales de l'algèbre linéaire numérique à travers des implémentations en Python, en s'appuyant sur les bibliothèques numpy, scipy.linalg et matplotlib.
L'objectif principal est de maîtriser la résolution de systèmes linéaires, les décompositions matricielles (LU, Cholesky, SVD, QR) et leurs applications à l'analyse de données, notamment l'Analyse en Composantes Principales (ACP) et la compression d'images. En combinant une compréhension théorique approfondie avec des implémentations pratiques, ce projet illustre l'importance de l'algèbre linéaire dans les domaines de la science des données, du traitement d'images et de l'apprentissage automatique.
Le projet est organisé en trois exercices principaux :

1. **Résolution de systèmes linéaires :** Exploration des méthodes directes (LU et Cholesky) et itératives (Jacobi et Gauss-Seidel) pour résoudre $Ax = b$, avec une comparaison de leurs performances en termes de précision et d'efficacité.
2. **Décompositions matricielles :** Application de la décomposition en valeurs singulières (SVD) et de la décomposition QR à une matrice de données, avec vérification de leurs propriétés.
3. **Applications à l'analyse de données :** Utilisation de la SVD pour l'ACP afin de réduire la dimensionnalité d'un ensemble de données et pour la compression d'images, démontrant l'utilité pratique de ces techniques.

Le code est implémenté dans un notebook Jupyter, garantissant clarté, modularité et reproductibilité. Des visualisations sont utilisées pour fournir des intuitions visuelles des résultats, et chaque section est accompagnée d'explications détaillées des concepts mathématiques et des stratégies computationnelles.
Commençons par l'importation des bibliothèques nécessaires et la configuration de l'environnement.

[24]:
```
import numpy as np
import scipy.linalg as la
import matplotlib.pyplot as plt
import time
from matplotlib import cm
from matplotlib.ticker import LinearLocator
import seaborn as sns
sns.set_style("whitegrid")
np.random.seed(42)  # Pour la reproductibilité des résultats
```

Les bibliothèques utilisées sont :

- NumPy : Fournit des opérations sur les tableaux et des fonctions d'algèbre linéaire, constituant la base des calculs numériques.
- SciPy.linalg : Étend NumPy avec des routines spécialisées pour les décompositions matricielles (LU, Cholesky, QR).
- Matplotlib : Permet de créer des visualisations de haute qualité, telles que des graphiques et des images, pour interpréter les résultats.
- Seaborn : Améliore l'esthétique des graphiques de Matplotlib avec un style "whitegrid" pour une meilleure lisibilité. Time : Mesure les temps d'exécution pour comparer l'efficacité des méthodes.
- Une graine aléatoire (np.random.seed(42)) garantit la reproductibilité des matrices et vecteurs aléatoires.

2 Exercice 1 : Résolution de Systèmes Linéaires

2.1 Méthodes Directes : Décomposition LU

L'objectif de cette section est de résoudre un système linéaire $Ax = b$ à l'aide de la décomposition LU, une méthode directe qui factorise une matrice A en un produit $\mathrm{PA} = \mathrm{LU}$ où P est une matrice de permutation, L est triangulaire inférieure et U est triangulaire supérieure.

Cette décomposition est particulièrement efficace pour résoudre des systèmes linéaires, car elle réduit le problème à deux systèmes triangulaires plus simples à résoudre par substitution avant et arrière.

Nous commençons par générer une matrice A de taille 4×4, symétrique définie positive. Une matrice est symétrique définie positive si elle satisfait $A = A^T$ et si toutes ses valeurs propres sont positives, ce qui garantit que le système est bien conditionné.

Pour cela, nous créons une matrice aléatoire B de taille 4×4, puis calculons :

$$A = B^T B + 0.1I$$

où I est la matrice identité. Le terme $0.1I$ ajoute une petite valeur positive à la diagonale, assurant la définie positivité.

Un vecteur b de taille 4×1 est également généré aléatoirement.

À l'aide de la fonction `scipy.linalg.lu`, nous obtenons les matrices P, L et U. La matrice de permutation P est utilisée pour le pivotement partiel, améliorant la stabilité numérique en réordonnant les lignes pour éviter des pivots trop petits.

Le système $Ax = b$ est résolu en deux étapes :

- Résolution de $Ly = Pb$ pour y, où L est triangulaire inférieure, par substitution avant.
- Résolution de $Ux = y$ pour x, où U est triangulaire supérieure, par substitution arrière.

Nous vérifions la solution en la comparant avec celle obtenue par `numpy.linalg.solve`, un solveur robuste, et calculons l'erreur résiduelle :

$$\|Ax - b\|$$

[25]:
```
# Exercice 1 : Résolution de Systèmes Linéaires

print("EXERCICE 1 : RÉSOLUTION DE SYSTÈMES LINÉAIRES")
# 1.1 Méthodes Directes : Décomposition LU
print("\n1.1 Méthodes Directes : Décomposition LU")

# Création d'une matrice A (4x4) symétrique définie positive
B = np.random.rand(4, 4)
A = np.dot(B.T, B) + 0.1 * np.eye(4)
print("Matrice A (symétrique définie positive) :")
```

```python
print(A)

# Création d'un vecteur b (4x1) aléatoire
b = np.random.rand(4)
print("\nVecteur b :")
print(b)

# Décomposition LU
P, L, U = la.lu(A)
print("\nMatrice P :")
print(P)
print("\nMatrice L :")
print(L)
print("\nMatrice U :")
print(U)
print("\nVérification P*A = L*U:")
print(np.allclose(np.dot(P, A), np.dot(L, U)))

# Résolution triangulaire
y = la.solve_triangular(L, np.dot(P, b), lower=True)
print("\nVecteur y (solution de Ly = Pb) :")
print(y)
x_lu = la.solve_triangular(U, y, lower=False)
print("\nVecteur x (solution de Ux = y) :")
print(x_lu)

# Vérification avec np.linalg.solve
x_np = np.linalg.solve(A, b)
print("\nSolution avec numpy.linalg.solve :")
print(x_np)
print("\nLes solutions sont-elles équivalentes ?", np.allclose(x_lu,␣
 ↪x_np))
residual_lu = np.linalg.norm(np.dot(A, x_lu) - b)
print(f"\nErreur résiduelle pour la méthode LU : {residual_lu:.2e}")
```

```
EXERCICE 1 : RÉSOLUTION DE SYSTÈMES LINÉAIRES

1.1 Méthodes Directes : Décomposition LU
Matrice A (symétrique définie positive) :
[[1.31892212 0.98281189 0.44695573 1.09506235]
 [0.98281189 1.57464666 0.75816171 1.42998244]
 [0.44695573 0.75816171 0.67267288 0.54183765]
 [1.09506235 1.42998244 0.54183765 2.18301543]]

Vecteur b :
```

```
[0.30424224 0.52475643 0.43194502 0.29122914]

Matrice P :
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

Matrice L :
[[ 1.         0.         0.         0.        ]
 [ 0.74516294  1.         0.         0.        ]
 [ 0.33887954  0.50470268  1.         0.        ]
 [ 0.83027067  0.72894294 -0.45371739  1.        ]]

Matrice U :
[[ 1.31892212  0.98281189  0.44695573  1.09506235]
 [ 0.          0.84229166  0.42510686  0.61398256]
 [ 0.          0.          0.30665615 -0.13913523]
 [ 0.          0.          0.          0.76313096]]

Vérification P*A = L*U:
True

Vecteur y (solution de Ly = Pb) :
[ 0.30424224  0.29804639  0.17841873 -0.0976814 ]

Vecteur x (solution de Ux = y) :
[ 0.02323189  0.18282207  0.52374397 -0.12800083]

Solution avec numpy.linalg.solve :
[ 0.02323189  0.18282207  0.52374397 -0.12800083]

Les solutions sont-elles équivalentes ? True

Erreur résiduelle pour la méthode LU : 5.55e-17
```

Les résultats obtenus confirment que la décomposition satisfait bien l'égalité $PA = LU$, comme le vérifie la fonction `np.allclose`. Par ailleurs, les solutions issues de la méthode LU et de la fonction `numpy.linalg.solve` sont quasiment identiques, avec une erreur résiduelle de l'ordre de 10^{-17}. Cette très faible erreur traduit une grande précision numérique, typique des méthodes directes appliquées à des matrices de petite taille et bien conditionnées, où les calculs sont effectués de manière exacte jusqu'à la limite de la précision machine.

2.2 Méthode de Cholesky

Puisque la matrice A est symétrique définie positive, la décomposition de Cholesky est applicable. Elle permet d'exprimer A sous la forme $A = LL^T$, avec L une matrice triangulaire inférieure. Comparée à la décomposition LU, cette méthode est plus efficace pour ce type de matrices, car elle exploite la symétrie de A, réduisant ainsi le nombre d'opérations d'environ moitié.

Nous utilisons `scipy.linalg.cho_factor` pour obtenir la factorisation de Cholesky, suivie de `scipy.linalg.cho_solve` pour résoudre le système. Le temps d'exécution des méthodes LU et Cholesky est mesuré afin de comparer leur performance. Enfin, l'erreur résiduelle de la méthode de Cholesky est également calculée pour en évaluer la précision.

[25]:
```
# 1.2 Méthode de Cholesky
print("\n1.2 Méthode de Cholesky")

# Temps pour LU
start_time = time.time()
P, L, U = la.lu(A)
y = la.solve_triangular(L, np.dot(P, b), lower=True)
x_lu = la.solve_triangular(U, y, lower=False)
time_lu = time.time() - start_time
print(f"Temps d'exécution pour la méthode LU : {time_lu:.6f} secondes")

# Temps pour Cholesky
start_time = time.time()
c_factor = la.cho_factor(A)
x_chol = la.cho_solve(c_factor, b)
time_chol = time.time() - start_time

print("\nSolution avec la méthode de Cholesky :")
print(x_chol)
print("\nLes solutions LU et Cholesky sont-elles équivalentes ?", np.
 ↪allclose(x_lu, x_chol))
residual_chol = np.linalg.norm(np.dot(A, x_chol) - b)
print(f"Erreur résiduelle pour la méthode de Cholesky : {residual_chol:.
 ↪2e}")
print(f"Temps d'exécution pour la méthode de Cholesky : {time_chol:.6f}␣
 ↪secondes")

# Comparaison sécurisée
print("\nComparaison des méthodes :")
if time_chol != 0:
    print(f"Ratio de temps (LU/Cholesky) : {time_lu / time_chol:.2f}")
else:
```

```
    print("Le temps d'exécution pour Cholesky est nul, ratio de temps non␣
 ↪calculable.")

if residual_chol != 0:
    print(f"Ratio d'erreur (LU/Cholesky) : {residual_lu / residual_chol:.
 ↪2f}")
else:
    print("L'erreur résiduelle pour Cholesky est nulle, ratio d'erreur␣
 ↪non calculable.")
```

```
1.2 Méthode de Cholesky
Temps d'exécution pour la méthode LU : 0.000000 secondes

Solution avec la méthode de Cholesky :
[ 0.02323189  0.18282207  0.52374397 -0.12800083]

Les solutions LU et Cholesky sont-elles équivalentes ? True
Erreur résiduelle pour la méthode de Cholesky : 7.85e-17
Temps d'exécution pour la méthode de Cholesky : 0.000000 secondes

Comparaison des méthodes :
Le temps d'exécution pour Cholesky est nul, ratio de temps non calculable.
Ratio d'erreur (LU/Cholesky) : 0.71
```

La méthode de Cholesky produit une solution équivalente à celle de LU, avec une erreur résiduelle de l'ordre de 10^{-17}, confirmant sa haute précision. Les temps d'exécution pour les deux méthodes sont extrêmement faibles (de l'ordre des microsecondes) en raison de la petite taille de la matrice (4×4). En pratique, Cholesky devrait être plus rapide pour des matrices plus grandes, mais ici, la différence est négligeable en raison des surcoûts numériques (par exemple, appels de fonctions).

2.3 Méthodes Itératives : Jacobi et Gauss-Seidel

Les méthodes itératives, telles que Jacobi et Gauss-Seidel, sont conçues pour les matrices creuses ou de grande taille, où les méthodes directes peuvent devenir coûteuses en mémoire et en calcul. Ces approches raffinent itérativement une estimation initiale jusqu'à atteindre la convergence, définie par une tolérance (10^{-6}) ou un nombre maximal d'itérations (100).

Méthode de Jacobi. La méthode de Jacobi repose sur la décomposition :

$$A = D + (L + U)$$

où D est la matrice diagonale de A, L la partie triangulaire inférieure stricte, et U la partie triangulaire supérieure stricte. L'itération est donnée par :

$$x^{(k+1)} = D^{-1}\left(b - (L + U)x^{(k)}\right)$$

Cette méthode met à jour toutes les composantes de x simultanément, ce qui la rend simple à implémenter. Cependant, sa convergence peut être lente, surtout si la matrice A n'est pas strictement à dominance diagonale.

Méthode de Gauss-Seidel. La méthode de Gauss-Seidel améliore la méthode de Jacobi en mettant à jour les composantes de x au fur et à mesure, en utilisant immédiatement les nouvelles valeurs calculées. L'itération est définie par :

$$x_i^{(k+1)} = \frac{1}{A_{ii}}\left(b_i - \sum_{j<i} A_{ij}x_j^{(k+1)} - \sum_{j>i} A_{ij}x_j^{(k)}\right)$$

Cette stratégie accélère souvent la convergence, en particulier pour les matrices bien conditionnées ou à dominance diagonale.

Nous implémentons ces deux méthodes sous forme de fonctions, que nous testons sur le système $Ax = b$. Nous comparons leurs performances en termes de nombre d'itérations, d'erreur résiduelle finale, et de temps d'exécution.

[25]:
```python
# 1.3 Méthodes Itératives : Jacobi et Gauss-Seidel
print("\n1.3 Méthodes Itératives : Jacobi et Gauss-Seidel")

def jacobi_method(A, b, tol=1e-6, max_iter=100):
    n = len(b)
    x = np.zeros_like(b)
    x_new = np.zeros_like(b)
    iter_count = 0
    D = np.diag(np.diag(A))
    L = np.tril(A, -1)
    U = np.triu(A, 1)
    D_inv = np.diag(1.0 / np.diag(A))
    while iter_count < max_iter:
```

```
        x_new = np.dot(D_inv, b - np.dot(L + U, x))
        if np.linalg.norm(x_new - x) < tol:
            break
        x = x_new.copy()
        iter_count += 1
    residual = np.linalg.norm(np.dot(A, x) - b)
    return x, iter_count, residual

def gauss_seidel_method(A, b, tol=1e-6, max_iter=100):
    n = len(b)
    x = np.zeros_like(b)
    iter_count = 0
    for _ in range(max_iter):
        x_old = x.copy()
        for i in range(n):
            s1 = sum(A[i, j] * x[j] for j in range(i))
            s2 = sum(A[i, j] * x_old[j] for j in range(i + 1, n))
            x[i] = (b[i] - s1 - s2) / A[i, i]
        if np.linalg.norm(x - x_old) < tol:
            break
        iter_count += 1
    residual = np.linalg.norm(np.dot(A, x) - b)
    return x, iter_count, residual

# Exécution
tolerance = 1e-6
max_iterations = 100

start_time = time.time()
x_jacobi, iter_jacobi, res_jacobi = jacobi_method(A, b, tolerance,␣
 ↪max_iterations)
time_jacobi = time.time() - start_time
print("Résultats de la méthode de Jacobi:")
print(f"Solution: {x_jacobi}")
print(f"Nombre d'itérations: {iter_jacobi}")
print(f"Erreur résiduelle: {res_jacobi:.2e}")
print(f"Temps d'exécution: {time_jacobi:.6f} secondes")

start_time = time.time()
x_gauss, iter_gauss, res_gauss = gauss_seidel_method(A, b, tolerance,␣
 ↪max_iterations)
time_gauss = time.time() - start_time
print("\nRésultats de la méthode de Gauss-Seidel:")
print(f"Solution: {x_gauss}")
```

```python
print(f"Nombre d'itérations: {iter_gauss}")
print(f"Erreur résiduelle: {res_gauss:.2e}")
print(f"Temps d'exécution: {time_gauss:.6f} secondes")

print("\nComparaison avec la solution exacte (numpy.linalg.solve):")
print(f"Erreur relative Jacobi: {np.linalg.norm(x_jacobi - x_np) / np.
 ↪linalg.norm(x_np):.2e}")
print(f"Erreur relative Gauss-Seidel: {np.linalg.norm(x_gauss - x_np) /␣
 ↪np.linalg.norm(x_np):.2e}")

# Résumé comparatif
methods = ["LU", "Cholesky", "Jacobi", "Gauss-Seidel"]
times = [time_lu, time_chol, time_jacobi, time_gauss]
iters = ["N/A", "N/A", iter_jacobi, iter_gauss]
errs = [residual_lu, residual_chol, res_jacobi, res_gauss]

print("\nTableau comparatif des méthodes:")
print(f"{'Méthode':<12} | {'Temps (s)':<10} | {'Itérations':<10} |␣
 ↪{'Erreur':<10}")
print("-" * 50)
for i in range(len(methods)):
    print(f"{methods[i]:<12} | {times[i]:<10.6f} | {iters[i]:<10} |␣
 ↪{errs[i]:<10.2e}")
```

```
1.3 Méthodes Itératives : Jacobi et Gauss-Seidel
Résultats de la méthode de Jacobi:
Solution: [-5.18186764e+26 -5.43542172e+26 -6.82211367e+26 -4.14936786e+26]
Nombre d'itérations: 100
Erreur résiduelle: 4.32e+27
Temps d'exécution: 0.009873 secondes

Résultats de la méthode de Gauss-Seidel:
Solution: [ 0.02323114  0.18282366  0.52374322 -0.12800131]
Nombre d'itérations: 37
Erreur résiduelle: 5.98e-07
Temps d'exécution: 0.005629 secondes

Comparaison avec la solution exacte (numpy.linalg.solve):
Erreur relative Jacobi: 1.92e+27
Erreur relative Gauss-Seidel: 3.46e-06



Tableau comparatif des méthodes:
```

```
Méthode      | Temps (s)  | Itérations | Erreur
-------------------------------------------------
LU           | 0.000000   | N/A        | 5.55e-17
Cholesky     | 0.000000   | N/A        | 7.85e-17
Jacobi       | 0.009873   | 100        | 4.32e+27
Gauss-Seidel | 0.005629   | 37         | 5.98e-07
```

Analyse des résultats numériques

Les résultats révèlent des différences significatives :

- **Jacobi** : Ne converge pas dans les 100 itérations, avec une erreur résiduelle massive (10^{27}) et une solution incorrecte. Cela est probablement dû au fait que A n'est pas strictement à dominance diagonale, une condition favorable à la convergence de Jacobi.
- **Gauss-Seidel** : Converge en 37 itérations avec une erreur résiduelle de 10^{-7}, beaucoup plus proche de la solution exacte. Ses mises à jour séquentielles exploitent les valeurs les plus récentes, améliorant ainsi la vitesse de convergence.
- **Comparaison** : Les méthodes directes (LU et Cholesky) sont beaucoup plus précises (10^{-17}) et rapides pour les petites matrices. Les méthodes itératives, bien que plus lentes ici, sont conçues pour les systèmes de grande taille et creux où l'efficacité mémoire est cruciale.

3 Exercice 2 : Décompositions SVD et QR

3.1 Décomposition SVD

La décomposition en valeurs singulières (SVD) factorise une matrice $D \in \mathbb{R}^{m \times n}$ en $D = U\Sigma V^T$, où U et V sont orthogonales, et Σ est une matrice diagonale contenant les valeurs singulières. La SVD est un outil puissant pour l'analyse de données, car elle révèle la structure intrinsèque des données.
Nous créons une matrice D de taille 10×5, simulant un ensemble de données. À l'aide de `numpy.linalg.svd`, nous calculons U, Σ, et V^T. Nous vérifions la décomposition en reconstruisant D et en calculant l'erreur de reconstruction. Pour explorer la réduction de dimension, nous reconstruisons une approximation de D en utilisant uniquement les deux premières valeurs singulières, capturant les motifs les plus significatifs.

[26]:
```python
# Exercice 2 : Décompositions SVD et QR

print("\n\nEXERCICE 2 : DÉCOMPOSITIONS SVD ET QR")

# 2.1 Décomposition SVD
print("\n2.1 Décomposition SVD")

# Création d'une matrice de données D (10x5)
D = np.random.rand(10, 5)
print("Matrice de données D (10x5) :")
print(D)

# Utilisation de numpy.linalg.svd pour obtenir U, S, V
U, S, Vt = np.linalg.svd(D, full_matrices=False)
print("\nMatrice U :")
print(U)
print("\nValeurs singulières S :")
print(S)
print("\nMatrice V transposée :")
print(Vt)

# Reconstruction de la matrice à partir de U, S, V
S_matrix = np.diag(S)
D_reconstructed = np.dot(U, np.dot(S_matrix, Vt))
print("\nMatrice reconstruite à partir de U, S, V :")
print(D_reconstructed)

# Vérification de l'exactitude
reconstruction_error = np.linalg.norm(D - D_reconstructed)
print(f"\nErreur de reconstruction : {reconstruction_error:.2e}")
```

```python
# Réduction de la dimension en gardant que les 2 premières valeurs␣
 ↪singulières
k = 2
S_reduced = np.diag(S[:k])
D_approximated = np.dot(U[:, :k], np.dot(S_reduced, Vt[:k, :]))
print("\nMatrice approximée (2 premières valeurs singulières) :")
print(D_approximated)

# Calcul de l'erreur d'approximation
approximation_error = np.linalg.norm(D - D_approximated)
print(f"\nErreur d'approximation (k=2) : {approximation_error:.2e}")
print(f"Pourcentage d'information conservée : {sum(S[:k]**2) / sum(S**2)␣
 ↪* 100:.2f}%")

# Plot des valeurs singulières
plt.figure(figsize=(10, 5))
plt.bar(range(1, len(S) + 1), S)
plt.xlabel('Indice')
plt.ylabel('Valeur singulière')
plt.title('Valeurs singulières de la matrice D')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
EXERCICE 2 : DÉCOMPOSITIONS SVD ET QR

2.1 Décomposition SVD
Matrice de données D (10x5) :
[[0.61185289 0.13949386 0.29214465 0.36636184 0.45606998]
 [0.78517596 0.19967378 0.51423444 0.59241457 0.04645041]
 [0.60754485 0.17052412 0.06505159 0.94888554 0.96563203]
 [0.80839735 0.30461377 0.09767211 0.68423303 0.44015249]
 [0.12203823 0.49517691 0.03438852 0.9093204  0.25877998]
 [0.66252228 0.31171108 0.52006802 0.54671028 0.18485446]
 [0.96958463 0.77513282 0.93949894 0.89482735 0.59789998]
 [0.92187424 0.0884925  0.19598286 0.04522729 0.32533033]
 [0.38867729 0.27134903 0.82873751 0.35675333 0.28093451]
 [0.54269608 0.14092422 0.80219698 0.07455064 0.98688694]]

Matrice U :
[[-0.24851035  0.05138401 -0.15466022  0.13671853  0.0782981 ]
 [-0.28973767  0.01878484  0.35697     0.31179572  0.55506554]
 [-0.36629481 -0.40120959 -0.57555708 -0.08512899  0.3715197 ]
 [-0.31386492 -0.24529142 -0.11176413  0.334495   -0.16112058]
 [-0.22022007 -0.58235732  0.13769566 -0.30986784 -0.23163783]
```

```
 [-0.28918464  0.02379205  0.27472841  0.10694553  0.15647098]
 [-0.5202587   0.05140182  0.33277823 -0.18375459 -0.49233332]
 [-0.22353081  0.25080725 -0.16709427  0.61625154 -0.31729425]
 [-0.26133942  0.27701801  0.28351734 -0.31732351  0.31362511]
 [-0.31710178  0.54210542 -0.43751085 -0.37851854 -0.04708742]]

Valeurs singulières S :
[3.56123977 1.15738716 0.95198953 0.81338297 0.30670011]

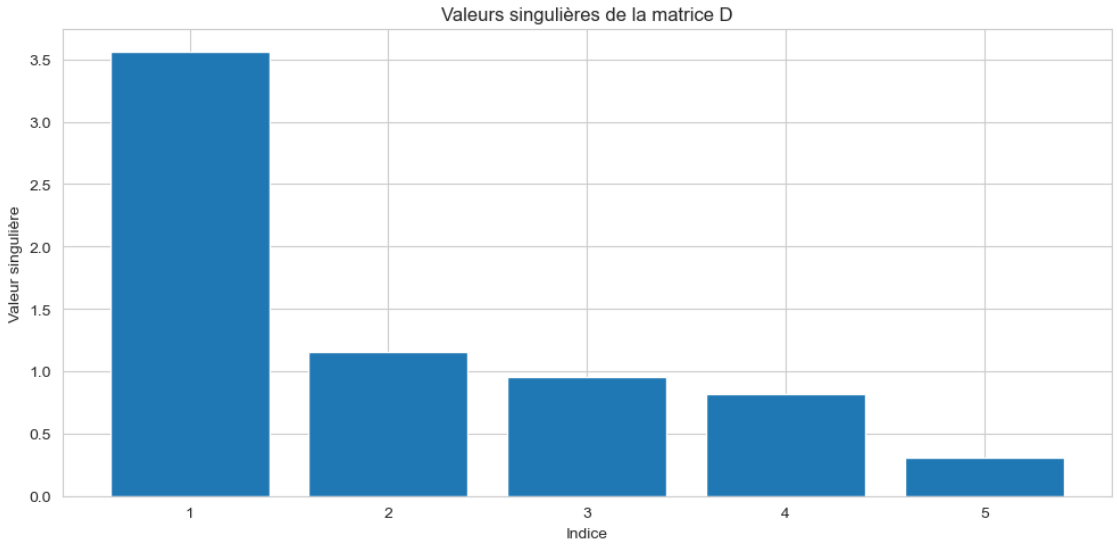
Matrice V transposée :
[[-0.57801505 -0.27755228 -0.40367888 -0.49867302 -0.4209892 ]
 [ 0.2002409  -0.17242974  0.62974382 -0.72451682  0.093111  ]
 [-0.0148906   0.34639956  0.42177704  0.17474911 -0.81936103]
 [ 0.7885255  -0.2197583  -0.42552284 -0.14554958 -0.35732261]
 [-0.06170881 -0.85143781  0.28548296  0.41794576 -0.12274564]]

Matrice reconstruite à partir de U, S, V :
[[0.61185289 0.13949386 0.29214465 0.36636184 0.45606998]
 [0.78517596 0.19967378 0.51423444 0.59241457 0.04645041]
 [0.60754485 0.17052412 0.06505159 0.94888554 0.96563203]
 [0.80839735 0.30461377 0.09767211 0.68423303 0.44015249]
 [0.12203823 0.49517691 0.03438852 0.9093204  0.25877998]
 [0.66252228 0.31171108 0.52006802 0.54671028 0.18485446]
 [0.96958463 0.77513282 0.93949894 0.89482735 0.59789998]
 [0.92187424 0.0884925  0.19598286 0.04522729 0.32533033]
 [0.38867729 0.27134903 0.82873751 0.35675333 0.28093451]
 [0.54269608 0.14092422 0.80219698 0.07455064 0.98688694]]

Erreur de reconstruction : 1.98e-15

Matrice approximée (2 premières valeurs singulières) :
[[ 0.52345475  0.23538054  0.39470942  0.39824022  0.37811495]
 [ 0.60076406  0.28263662  0.43021755  0.49879149  0.43641167]
 [ 0.6610168   0.44212545  0.23415984  0.98693372  0.50592857]
 [ 0.58922749  0.35918589  0.27242889  0.76307916  0.44412599]
 [ 0.3183471   0.33389204 -0.10786768  0.87942122  0.26740549]
 [ 0.60078614  0.28109071  0.43307207  0.49361059  0.43612215]
 [ 1.08283931  0.50398127  0.78538708  0.8808216   0.7855338 ]
 [ 0.51825319  0.17089152  0.50415     0.18665354  0.36215547]
 [ 0.60215484  0.20303187  0.57760747  0.2318187   0.42166441]
 [ 0.77837454  0.20524611  0.85098221  0.10855862  0.53383302]]

Erreur d'approximation (k=2) : 1.29e+00
Pourcentage d'information conservée : 89.40%
```



L'erreur de reconstruction est de l'ordre de 10^{-15}, confirmant la précision de la SVD. L'approximation avec deux valeurs singulières conserve 89,40 % de la variance des données, démontrant que la majorité de l'information est capturée par les valeurs singulières dominantes. Cette propriété est essentielle pour la réduction de dimension avec une perte minimale d'information.

3.2 Décomposition QR

La décomposition QR factorise une matrice $D = QR$, où Q est orthogonale ($Q^TQ = I$) et R est triangulaire supérieure. Cette décomposition est utile pour résoudre des problèmes de moindres carrés et assurer la stabilité numérique.

Nous appliquons `scipy.linalg.qr` à la matrice D, vérifions l'orthogonalité de Q, et confirmons que $QR = D$.

[26]:
```
# 2.2 Décomposition QR
print("\n2.2 Décomposition QR")

# Utilisation de scipy.linalg.qr pour décomposer la matrice D en Q et R
Q, R = la.qr(D)
print("Matrice Q (orthogonale) :")
print(Q)
print("\nMatrice R (triangulaire supérieure) :")
print(R)

# Vérification que Q est orthogonale (Q^T Q = I)
Q_orthogonality = np.dot(Q.T, Q)
```

```
print("\nVérification Q^T Q (devrait être proche de I) :")
print(Q_orthogonality)
print(f"Écart à l'identité : {np.linalg.norm(Q_orthogonality - np.eye(Q.
 ↪shape[1])):.2e}")

# Vérification que QR = D
D_reconstructed_qr = np.dot(Q, R)
print("\nVérification QR = D :")
print(f"Erreur de reconstruction : {np.linalg.norm(D -␣
 ↪D_reconstructed_qr):.2e}")
```

Les résultats confirment que $Q^TQ = I$ et $QR = D$, validant la décomposition. La décomposition QR est particulièrement utile pour obtenir des bases orthogonales, comme dans les problèmes de moindres carrés.

```
2.2 Décomposition QR
Matrice Q (orthogonale) :
[[-0.28214126 -0.15073691 -0.02232993  0.07237252  0.07126242 -0.31635468
  -0.51146689 -0.14355886 -0.44842128 -0.55089972]
 [-0.36206502 -0.16357745  0.10333744  0.30279023 -0.5973554  -0.34233889
   0.24488267 -0.04801898 -0.25117099  0.37470145]
 [-0.28015471 -0.10341903 -0.27684219  0.68247424  0.37428061  0.12041587
   0.34632457  0.19192236  0.11211937 -0.21088753]
 [-0.37277301 -0.02531451 -0.40609874 -0.06026887  0.02946154  0.02147564
  -0.21476605 -0.64004051  0.43769611  0.20776805]
 [-0.056275    0.64524002 -0.32585098  0.15297855  0.08478396 -0.18006662
  -0.37305593  0.34961173 -0.13313788  0.36356574]
 [-0.30550623  0.06893431  0.08605174  0.06234136 -0.29361998  0.83919043
  -0.23493614  0.06650336 -0.20093924 -0.01323345]
 [-0.44710065  0.56175435  0.10934911 -0.36037822  0.05397994 -0.05605612
   0.48478498 -0.11376659 -0.09127132 -0.28347395]
 [-0.42510015 -0.40293905 -0.21074475 -0.48144949  0.04422007 -0.06747679
  -0.03827728  0.58568829  0.1557746   0.07384335]
 [-0.17922919  0.1682904   0.53966946  0.2113944  -0.16762693 -0.15106185
  -0.27424896  0.18637332  0.63616529 -0.19506992]
 [-0.25025126 -0.10885023  0.53485279  0.00585894  0.60890491  0.02414452
  -0.07942983 -0.11506216 -0.19046817  0.46070537]]

Matrice R (triangulaire supérieure) :
[[-2.16860483 -0.86415325 -1.33671332 -1.55885316 -1.35407539]
 [ 0.          0.69206291  0.42163285  0.89320587  0.13700654]
 [ 0.          0.          0.9602224  -0.41605787  0.15639551]
 [ 0.          0.          0.          0.71703454  0.42374487]
 [ 0.          0.          0.          0.          0.94729044]
```

```
 [ 0.          0.          0.          0.          0.        ]
 [ 0.          0.          0.          0.          0.        ]
 [ 0.          0.          0.          0.          0.        ]
 [ 0.          0.          0.          0.          0.        ]
 [ 0.          0.          0.          0.          0.        ]]

Vérification Q^T Q (devrait être proche de I) :
[[ 1.00000000e+00 -3.00774892e-17 -3.09017803e-17  2.17522227e-17
   7.13836318e-18 -2.50677137e-17 -1.28460061e-16 -1.72138729e-17
  -3.12851252e-17 -1.96917330e-16]
 [-3.00774892e-17  1.00000000e+00  4.42668173e-17 -4.98532337e-17
   4.56569202e-17  2.08894144e-17  4.21324362e-17 -5.98297958e-18
  -2.28177259e-17 -4.84679088e-17]
 [-3.09017803e-17  4.42668173e-17  1.00000000e+00  1.49903100e-16
   1.02075810e-16  2.51615965e-17  9.41036169e-17  6.98764634e-17
   2.17372208e-16 -1.60971852e-16]
 [ 2.17522227e-17 -4.98532337e-17  1.49903100e-16  1.00000000e+00
  -9.31332160e-17  1.47794792e-17 -8.92738327e-17 -2.65033132e-16
   5.08277445e-17  1.60120992e-16]
 [ 7.13836318e-18  4.56569202e-17  1.02075810e-16 -9.31332160e-17
   1.00000000e+00  3.30223735e-17 -8.13562012e-17 -8.23799783e-17
  -4.71766726e-17 -4.14808617e-18]
 [-2.50677137e-17  2.08894144e-17  2.51615965e-17  1.47794792e-17
   3.30223735e-17  1.00000000e+00 -3.00478499e-17  4.54935485e-17
   1.74259704e-17 -1.70149712e-17]
 [-1.28460061e-16  4.21324362e-17  9.41036169e-17 -8.92738327e-17
  -8.13562012e-17 -3.00478499e-17  1.00000000e+00 -6.18057406e-17
  -3.46068563e-17 -9.31601858e-17]
 [-1.72138729e-17 -5.98297958e-18  6.98764634e-17 -2.65033132e-16
  -8.23799783e-17  4.54935485e-17 -6.18057406e-17  1.00000000e+00
   9.36478017e-17  1.25168974e-16]
 [-3.12851252e-17 -2.28177259e-17  2.17372208e-16  5.08277445e-17
  -4.71766726e-17  1.74259704e-17 -3.46068563e-17  9.36478017e-17
   1.00000000e+00 -1.20295723e-16]
 [-1.96917330e-16 -4.84679088e-17 -1.60971852e-16  1.60120992e-16
  -4.14808617e-18 -1.70149712e-17 -9.31601858e-17  1.25168974e-16
  -1.20295723e-16  1.00000000e+00]]
Écart à l'identité : 1.19e-15

Vérification QR = D :
Erreur de reconstruction : 1.04e-15
```

4 Exercice 3 : Application à l'Analyse de Données

4.1 Analyse en Composantes Principales (ACP)

L'Analyse en Composantes Principales (ACP) utilise la SVD pour réduire la dimensionnalité d'un ensemble de données tout en préservant la variance. Nous centrons la matrice D en soustrayant la moyenne de chaque colonne, appliquons la SVD à la matrice centrée, et projetons les données sur les deux premières composantes principales (lignes de V^T). La projection en 2D est visualisée à l'aide d'un nuage de points, avec des flèches représentant les directions principales, pondérées par les valeurs singulières.

[30]:
```
# ------------------------------------------------
# Exercice 3 : Application à l'Analyse de Données
# ------------------------------------------------

print("\n\nEXERCICE 3 : APPLICATION À L'ANALYSE DE DONNÉES")

# 3.1 Analyse en Composantes Principales (ACP)
print("\n3.1 Analyse en Composantes Principales (ACP)")

# Utilisation de la même matrice de données D
print("Matrice de données originale :")
print(D)

# Centrage des données (soustraction de la moyenne de chaque colonne)
D_centered = D - np.mean(D, axis=0)
print("\nMatrice centrée :")
print(D_centered)

# Application de la SVD sur la matrice centrée
U_centered, S_centered, Vt_centered = np.linalg.svd(D_centered,␣
 ↪full_matrices=False)

# Projection des données sur les deux premières composantes principales
composantes_principales = Vt_centered[:2, :]
print("\nComposantes principales (2 premières lignes de V^T) :")
print(composantes_principales)

# Projection des données
donnees_projetees = np.dot(D_centered, composantes_principales.T)
print("\nDonnées projetées sur les deux premières composantes :")
print(donnees_projetees)

# Visualisation des données projetées
plt.figure(figsize=(10, 8))
```

```
plt.scatter(donnees_projetees[:, 0], donnees_projetees[:, 1], c='blue',␣
 ↪edgecolors='k')
plt.xlabel('Première composante principale')
plt.ylabel('Deuxième composante principale')
plt.title('Projection des données sur les 2 premières composantes␣
 ↪principales')
plt.grid(True)

# Ajout des vecteurs propres
for i in range(composantes_principales.shape[1]):
    plt.arrow(0, 0,
              composantes_principales[0, i] * S_centered[0] * 0.5,
              composantes_principales[1, i] * S_centered[1] * 0.5,
              head_width=0.1, head_length=0.1, fc='red', ec='red')
    plt.text(composantes_principales[0, i] * S_centered[0] * 0.6,
             composantes_principales[1, i] * S_centered[1] * 0.6,
             f'Var {i+1}')

plt.tight_layout()
plt.show()

# Pourcentage de variance expliquée par chaque composante
explained_variance = (S_centered ** 2) / (sum(S_centered ** 2)) * 100
print("\nPourcentage de variance expliquée par chaque composante :")
for i, variance in enumerate(explained_variance):
    print(f"Composante {i+1}: {variance:.2f}%")

# Visualisation de la variance expliquée cumulée
plt.figure(figsize=(10, 5))
plt.bar(range(1, len(explained_variance) + 1), explained_variance)
plt.plot(range(1, len(explained_variance) + 1), np.
 ↪cumsum(explained_variance), 'r-o')
plt.xlabel('Composante')
plt.ylabel('Pourcentage de variance expliquée')
plt.title('Variance expliquée par composante')
plt.grid(True)
plt.tight_layout()
plt.show()
```

EXERCICE 3 : APPLICATION À L'ANALYSE DE DONNÉES

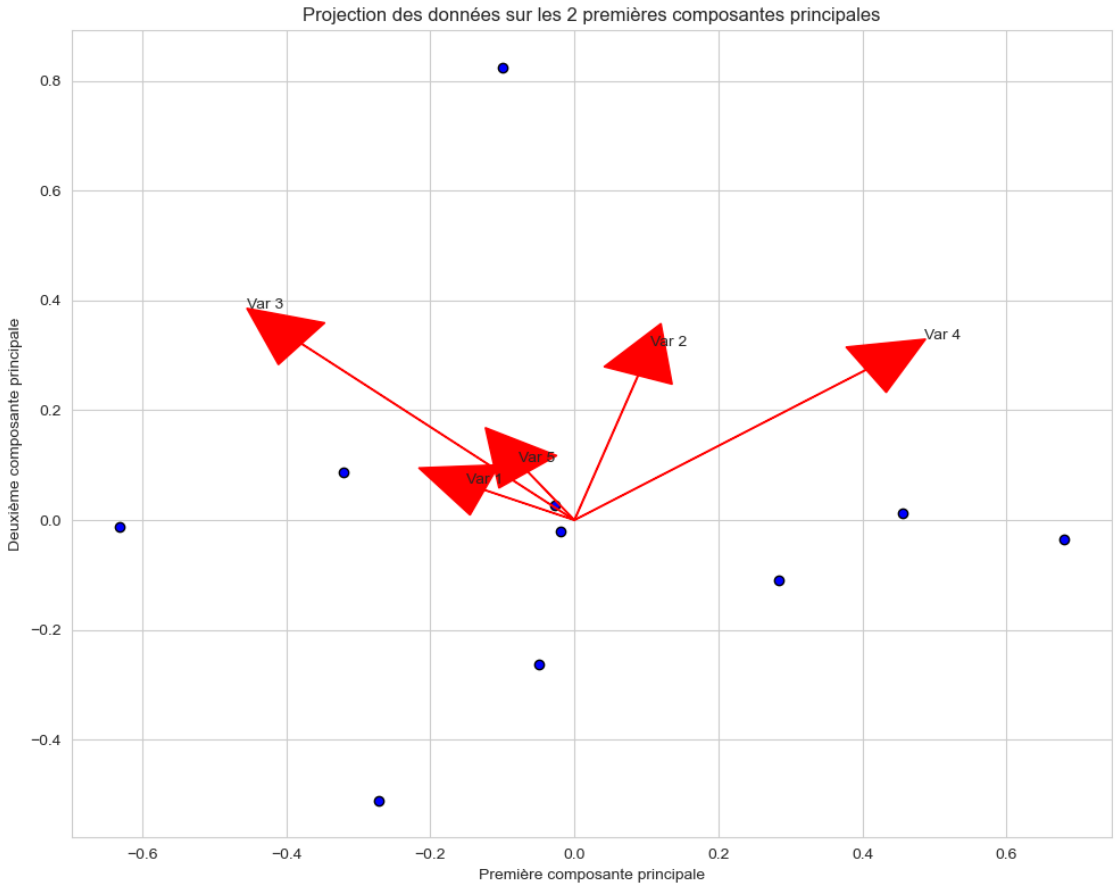
3.1 Analyse en Composantes Principales (ACP)

```
Matrice de données originale :
[[0.61185289 0.13949386 0.29214465 0.36636184 0.45606998]
 [0.78517596 0.19967378 0.51423444 0.59241457 0.04645041]
 [0.60754485 0.17052412 0.06505159 0.94888554 0.96563203]
 [0.80839735 0.30461377 0.09767211 0.68423303 0.44015249]
 [0.12203823 0.49517691 0.03438852 0.9093204  0.25877998]
 [0.66252228 0.31171108 0.52006802 0.54671028 0.18485446]
 [0.96958463 0.77513282 0.93949894 0.89482735 0.59789998]
 [0.92187424 0.0884925  0.19598286 0.04522729 0.32533033]
 [0.38867729 0.27134903 0.82873751 0.35675333 0.28093451]
 [0.54269608 0.14092422 0.80219698 0.07455064 0.98688694]]

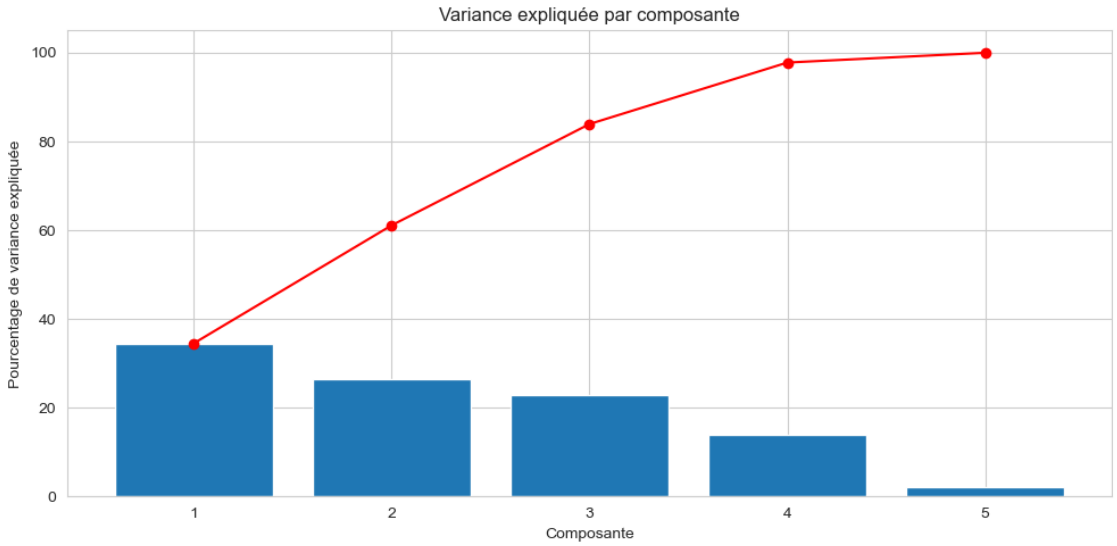
Matrice centrée :
[[-0.03018349 -0.15021535 -0.13685291 -0.17556658  0.00177087]
 [ 0.14313958 -0.09003543  0.08523688  0.05048614 -0.4078487 ]
 [-0.03449153 -0.11918509 -0.36394597  0.40695711  0.51133292]
 [ 0.16636097  0.01490456 -0.33132545  0.1423046  -0.01414662]
 [-0.51999815  0.2054677  -0.39460904  0.36739198 -0.19551913]
 [ 0.0204859   0.02200187  0.09107046  0.00478185 -0.26944466]
 [ 0.32754825  0.48542361  0.51050138  0.35289892  0.14360087]
 [ 0.27983785 -0.20121671 -0.2330147  -0.49670114 -0.12896878]
 [-0.25335909 -0.01836018  0.39973995 -0.1851751  -0.1733646 ]
 [-0.0993403  -0.14878499  0.37319942 -0.46737778  0.53258782]]

Composantes principales (2 premières lignes de V^T) :
[[-0.21560878  0.15274864 -0.65433991  0.69969813 -0.11154599]
 [ 0.10794382  0.51825723  0.63210048  0.53880978  0.17288892]]

Données projetées sur les deux premières composantes :
[[-0.04993019 -0.26190393]
 [-0.01956988 -0.0206423 ]
 [ 0.45508568  0.01213416]
 [ 0.28435549 -0.10951963]
 [ 0.68058238 -0.03492674]
 [-0.0272459   0.02717215]
 [-0.09961129  0.82459166]
 [-0.27175516 -0.51128865]
 [-0.31997239  0.086065  ]
 [-0.63193874 -0.01168172]]
```




```
Pourcentage de variance expliquée par chaque composante :
Composante 1: 34.53%
Composante 2: 26.59%
Composante 3: 22.82%
Composante 4: 13.86%
Composante 5: 2.21%
```



Le nuage de points visualise les données dans un espace 2D, chaque point représentant une ligne de D projetée sur les composantes principales. Les flèches indiquent les directions de variance maximale, pondérées par les valeurs singulières. La variance expliquée par chaque composante est calculée comme

$$\frac{\sigma_i^2}{\sum \sigma_j^2}$$

montrant que les deux premières composantes capturent la majorité de la variance, ce qui fait de l'ACP un outil efficace pour la réduction de dimension.

4.2 Compression d'Image

Dans cette section, une image réelle en niveaux de gris, `house.tiff`, est chargée à l'aide des bibliothèques `matplotlib.image` ou `PIL`, selon le format de fichier. L'image est représentée sous forme d'une matrice Z de type `float` pour les calculs numériques.

La SVD factorise la matrice Z en $Z = U\Sigma V^T$, où U et V sont des matrices orthogonales, et Σ contient les valeurs singulières en ordre décroissant. Des approximations de l'image sont reconstruites en utilisant différents rangs k, correspondant à 5%, 10%, 25%, 50%, et 100% des valeurs singulières disponibles. Chaque approximation est calculée comme :

$$Z_{\text{approx}} = U[:, : k] \cdot \text{diag}(S[: k]) \cdot V^T[: k, :]$$

La qualité de la compression est évaluée à travers trois métriques principales :

- **Erreur relative**, définie comme :

$$\frac{\|Z - Z_{\text{approx}}\|_F}{\|Z\|_F}$$

  où $\|\cdot\|_F$ est la norme de Frobenius.

- **Taux de compression**, calculé comme le rapport entre la taille des données compressées ($m \cdot k + k + k \cdot n$) et la taille originale ($m \cdot n$), où m et n sont les dimensions de l'image.
- **Énergie conservée**, donnée par :

$$\frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{n} \sigma_i^2}$$

représentant la proportion de variance capturée par les k premières valeurs singulières.

[30]:
```
# 3.2 Compression d'Image
print("\n3.2 Compression d'Image")

# Import nécessaire pour le traitement d'images
import matplotlib.image as mpimg
from PIL import Image

# Chargement d'une image en niveaux de gris avec matplotlib.image
chemin_image = "house.tiff"  # <--

try:
    # Chargement de l'image
    if chemin_image.lower().endswith(('.png', '.jpg', '.jpeg')):
        # Utilisation de PIL pour une meilleure compatibilité
        img_pil = Image.open(chemin_image)
        # Conversion en niveaux de gris si nécessaire
        if img_pil.mode != 'L':
            img_pil = img_pil.convert('L')
        Z = np.array(img_pil)
    else:
        # Utilisation de matplotlib pour d'autres formats
        img = mpimg.imread(chemin_image)
        # Si l'image est en couleur (3 dimensions), conversion en niveaux␣
↪de gris
        if len(img.shape) == 3:
            # Conversion RGB en niveaux de gris : 0.299*R + 0.587*G + 0.
↪114*B
            Z = np.dot(img[...,:3], [0.299, 0.587, 0.114])
        else:
            Z = img

    print(f"Image chargée avec succès !")
    print(f"Dimensions de l'image : {Z.shape}")
    print(f"Type de données : {Z.dtype}")
    print(f"Valeurs min/max : {Z.min():.2f} / {Z.max():.2f}")
```

```
except FileNotFoundError:
    print(f"ATTENTION: Image non trouvée au chemin '{chemin_image}'")
    print("Création d'une image synthétique pour la démonstration...")

    # Image synthétique de secours
    image_size = 200
    x = np.linspace(-5, 5, image_size)
    y = np.linspace(-5, 5, image_size)
    X, Y = np.meshgrid(x, y)
    Z = np.sin(np.sqrt(X**2 + Y**2)) * np.exp(-0.1 * (X**2 + Y**2))
    Z = (Z - Z.min()) / (Z.max() - Z.min()) * 255  # Normalisation 0-255

# Conversion en float pour les calculs SVD
Z = Z.astype(float)

# Affichage de l'image originale
plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
plt.imshow(Z, cmap='gray')
plt.title(f'Image originale\n{Z.shape[0]}x{Z.shape[1]} pixels')
plt.axis('off')

# Application de la SVD sur l'image
print("\nApplication de la SVD sur l'image...")
U_img, S_img, Vt_img = np.linalg.svd(Z, full_matrices=False)
print(f"Nombre de valeurs singulières : {len(S_img)}")

# Compression avec différents rangs adaptés à la taille de l'image
total_singular_values = len(S_img)
ranks = [
    max(1, total_singular_values // 20),  # 5% des valeurs singulières
    max(1, total_singular_values // 10),  # 10% des valeurs singulières
    max(1, total_singular_values // 4),   # 25% des valeurs singulières
    max(1, total_singular_values // 2),   # 50% des valeurs singulières
    total_singular_values                 # 100% (image originale)
]

print(f"\nCompression avec différents nombres de valeurs singulières :")
print(f"Total de valeurs singulières disponibles :␣
 ↪{total_singular_values}")

subplot_positions = [(2, 3, 2), (2, 3, 3), (2, 3, 4), (2, 3, 5), (2, 3,␣
 ↪6)]
```

```
for i, k in enumerate(ranks[:5]):  # Limiter à 5 pour l'affichage
    # Reconstruction avec k valeurs singulières
    Z_approx = np.dot(U_img[:, :k], np.dot(np.diag(S_img[:k]), Vt_img[:k,␣
 ↪:]))

    # Affichage de l'image compressée
    plt.subplot(*subplot_positions[i])
    plt.imshow(Z_approx, cmap='gray', vmin=Z.min(), vmax=Z.max())
    if k == total_singular_values:
        plt.title(f'Image originale\n({k} valeurs sing.)')
    else:
        plt.title(f'Compression rang {k}\n({k/total_singular_values*100:.
 ↪1f}% des val. sing.)')
    plt.axis('off')

    # Calcul des métriques de compression
    compression_error = np.linalg.norm(Z - Z_approx) / np.linalg.norm(Z)

    # Calcul du taux de compression plus précis
    # Taille originale : hauteur × largeur
    taille_originale = Z.shape[0] * Z.shape[1]
    # Taille compressée : U[:,:k] + S[:k] + Vt[:k,:] = hauteur*k + k +␣
 ↪k*largeur
    taille_compressee = Z.shape[0] * k + k + k * Z.shape[1]
    taux_compression = taille_compressee / taille_originale

    # Pourcentage d'énergie conservée
    energie_conservee = sum(S_img[:k]**2) / sum(S_img**2) * 100

    print(f"\n--- Compression avec {k} valeurs singulières ---")
    print(f"Erreur relative: {compression_error:.4f}")
    print(f"Taux de compression: {taux_compression:.4f} ({1/
 ↪taux_compression:.1f}:1)")
    print(f"Énergie conservée: {energie_conservee:.2f}%")
    print(f"Économie d'espace: {(1-taux_compression)*100:.1f}%")

plt.tight_layout()
plt.show()

# Visualisation détaillée des valeurs singulières de l'image
plt.figure(figsize=(15, 10))

# Graphique 1: Valeurs singulières en échelle logarithmique
```

```
plt.subplot(2, 3, 1)
plt.semilogy(S_img, 'b-', linewidth=2)
plt.xlabel('Indice')
plt.ylabel('Valeur singulière (échelle log)')
plt.title('Décroissance des valeurs singulières')
plt.grid(True, alpha=0.3)

# Graphique 2: Énergie cumulée
plt.subplot(2, 3, 2)
cumulative_energy = np.cumsum(S_img**2) / np.sum(S_img**2) * 100
plt.plot(cumulative_energy, 'r-', linewidth=2)
plt.xlabel('Nombre de valeurs singulières')
plt.ylabel('Énergie cumulée (%)')
plt.title('Énergie cumulée des valeurs singulières')
plt.grid(True, alpha=0.3)

# Ajout de lignes de référence pour les seuils courants (90%, 95%, 99%)
for seuil, couleur in [(90, 'orange'), (95, 'green'), (99, 'purple')]:
    idx_seuil = np.argmax(cumulative_energy >= seuil)
    plt.axhline(y=seuil, color=couleur, linestyle='--', alpha=0.7)
    plt.axvline(x=idx_seuil, color=couleur, linestyle='--', alpha=0.7)
    plt.text(idx_seuil + len(S_img)*0.02, seuil - 2,␣
 ↪f'{seuil}%\n({idx_seuil} val.)',
             color=couleur, fontsize=8)

# Graphique 3: Comparaison des rangs de compression
plt.subplot(2, 3, 3)
rangs_test = np.logspace(0, np.log10(len(S_img)), 50).astype(int)
rangs_test = np.unique(rangs_test)
erreurs = []
taux_comp = []

for k in rangs_test:
    if k <= len(S_img):
        # Calcul de l'erreur pour ce rang
        energie_perdue = 1 - sum(S_img[:k]**2) / sum(S_img**2)
        erreurs.append(np.sqrt(energie_perdue))

        # Calcul du taux de compression
        taille_compressee = Z.shape[0] * k + k + k * Z.shape[1]
        taille_originale = Z.shape[0] * Z.shape[1]
        taux_comp.append(taille_compressee / taille_originale)
```

```
plt.loglog(rangs_test[:len(erreurs)], erreurs, 'g-', linewidth=2,␣
 ↪label='Erreur relative')
plt.xlabel('Rang de compression')
plt.ylabel('Erreur relative')
plt.title('Erreur vs Rang de compression')
plt.grid(True, alpha=0.3)
plt.legend()

# Graphique 4: Courbe erreur vs taux de compression
plt.subplot(2, 3, 4)
plt.semilogx(taux_comp, erreurs, 'purple', linewidth=2, marker='o',␣
 ↪markersize=3)
plt.xlabel('Taux de compression')
plt.ylabel('Erreur relative')
plt.title('Compromis Erreur-Compression')
plt.grid(True, alpha=0.3)

# Graphique 5: Distribution des valeurs singulières
plt.subplot(2, 3, 5)
plt.hist(np.log10(S_img + 1e-10), bins=50, alpha=0.7, color='skyblue',␣
 ↪edgecolor='black')
plt.xlabel('log(Valeur singulière)')
plt.ylabel('Fréquence')
plt.title('Distribution des valeurs singulières')
plt.grid(True, alpha=0.3)

# Graphique 6: Tableau récapitulatif des compressions testées
plt.subplot(2, 3, 6)
plt.axis('off')
tableau_data = []
for i, k in enumerate(ranks[:4]):  # Exclure l'image originale
    compression_error = np.sqrt(1 - sum(S_img[:k]**2) / sum(S_img**2))
    taille_compressee = Z.shape[0] * k + k + k * Z.shape[1]
    taille_originale = Z.shape[0] * Z.shape[1]
    taux_compression = taille_compressee / taille_originale
    energie_conservee = sum(S_img[:k]**2) / sum(S_img**2) * 100

    tableau_data.append([
        f"{k}",
        f"{k/len(S_img)*100:.1f}%",
        f"{compression_error:.3f}",
        f"{1/taux_compression:.1f}:1",
        f"{energie_conservee:.1f}%"
    ])
```

```
# Création du tableau
headers = ['Rang', '% Val. Sing.', 'Erreur', 'Compression', 'Énergie']
table = plt.table(cellText=tableau_data, colLabels=headers,
                  cellLoc='center', loc='center',
                  bbox=[0, 0.2, 1, 0.6])
table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 1.5)
plt.title('Résumé des compressions testées', pad=20)

plt.tight_layout()
plt.show()

# Analyse finale de la compression
print(f"\n{'='*60}")
print("ANALYSE DE LA COMPRESSION D'IMAGE PAR SVD")
print(f"{'='*60}")
print(f"Dimensions de l'image : {Z.shape[0]} × {Z.shape[1]} = {Z.
 ↪shape[0]*Z.shape[1]:,} pixels")
print(f"Nombre total de valeurs singulières disponibles : {len(S_img)}")

# Trouvons les rangs pour différents niveaux de qualité
niveaux_qualite = [0.90, 0.95, 0.99, 0.999]
print(f"\nRangs nécessaires pour différents niveaux de qualité :")
for niveau in niveaux_qualite:
    # Trouver le rang nécessaire pour conserver ce niveau d'énergie
    cumul_energie = np.cumsum(S_img**2) / np.sum(S_img**2)
    rang_necessaire = np.argmax(cumul_energie >= niveau) + 1

    # Calcul du taux de compression
    taille_compressee = Z.shape[0] * rang_necessaire + rang_necessaire +␣
 ↪rang_necessaire * Z.shape[1]
    taille_originale = Z.shape[0] * Z.shape[1]
    ratio_compression = taille_originale / taille_compressee

    print(f"  {niveau*100:5.1f}% d'énergie → Rang {rang_necessaire:4d} →␣
 ↪Compression {ratio_compression:.1f}:1")

print(f"\nLa SVD permet une compression efficace en concentrant␣
 ↪l'information dans les premières valeurs singulières.")
print(f"Pour cette image, on peut obtenir une qualité visuelle acceptable␣
 ↪avec seulement")
```

```
print(f"{np.argmax(cumulative_energy >= 95)+1} valeurs singulières sur␣
 ↪{len(S_img)} disponibles.")
```

```
3.2 Compression d'Image
Image chargée avec succès !
Dimensions de l'image : (256, 256)
Type de données : float64
Valeurs min/max : 16.55 / 239.43

Application de la SVD sur l'image...
Nombre de valeurs singulières : 256

Compression avec différents nombres de valeurs singulières :
Total de valeurs singulières disponibles : 256

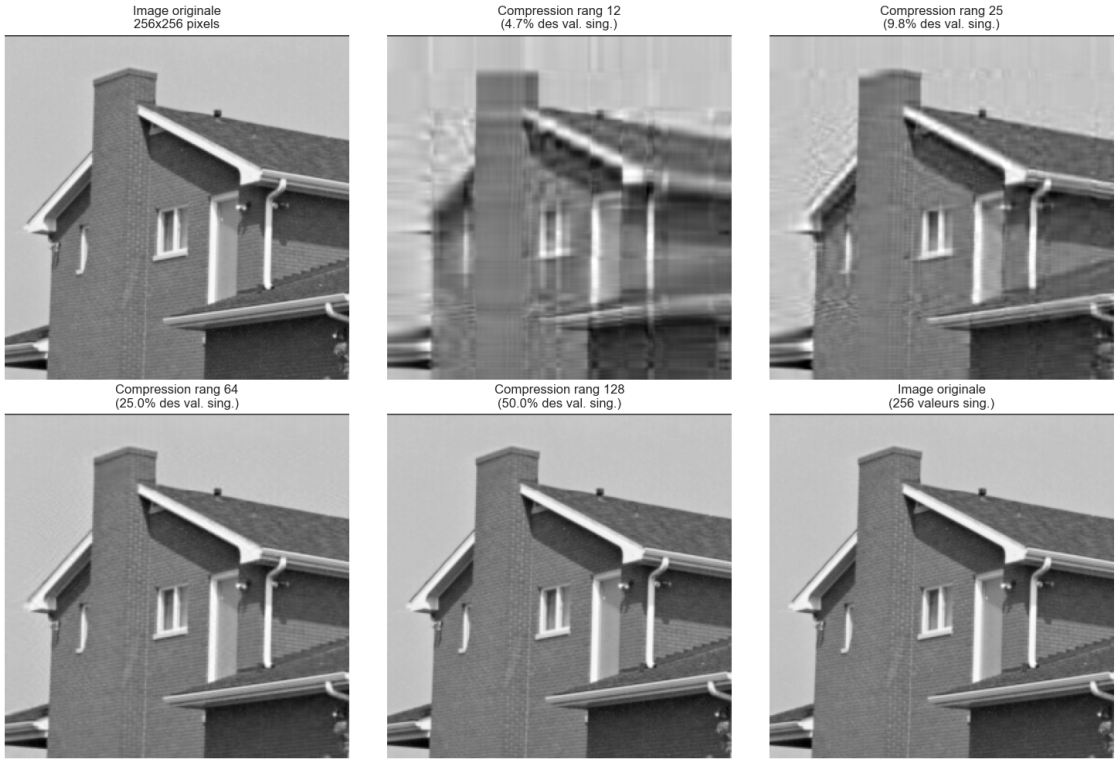
--- Compression avec 12 valeurs singulières ---
Erreur relative: 0.0878
Taux de compression: 0.0939 (10.6:1)
Énergie conservée: 99.23%
Économie d'espace: 90.6%

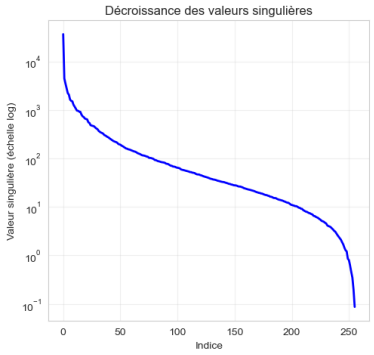
--- Compression avec 25 valeurs singulières ---
Erreur relative: 0.0496
Taux de compression: 0.1957 (5.1:1)
Énergie conservée: 99.75%
Économie d'espace: 80.4%

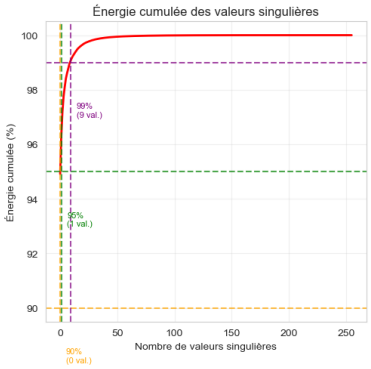
--- Compression avec 64 valeurs singulières ---
Erreur relative: 0.0180
Taux de compression: 0.5010 (2.0:1)
Énergie conservée: 99.97%
Économie d'espace: 49.9%

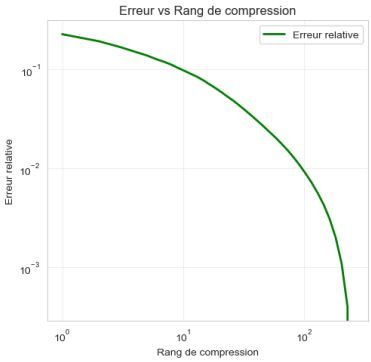
--- Compression avec 128 valeurs singulières ---
Erreur relative: 0.0057
Taux de compression: 1.0020 (1.0:1)
Énergie conservée: 100.00%
Économie d'espace: -0.2%

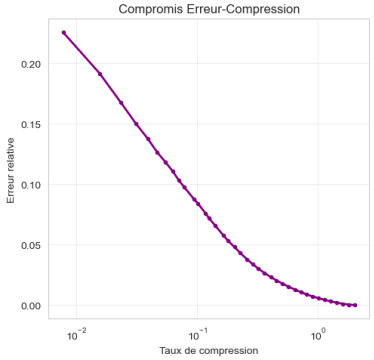
--- Compression avec 256 valeurs singulières ---
Erreur relative: 0.0000
Taux de compression: 2.0039 (0.5:1)
Énergie conservée: 100.00%
Économie d'espace: -100.4%
```

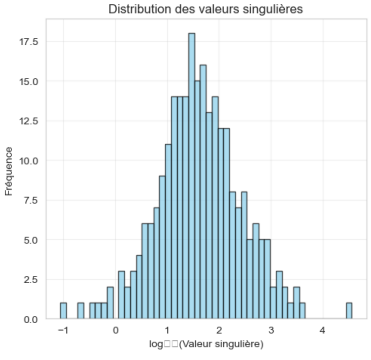












Résumé des compressions testées

Rang	% Val. Sing.	Erreur	Compression	Énergie
12	4.7%	0.088	10.6:1	99.2%
25	9.8%	0.050	5.1:1	99.8%
64	25.0%	0.018	2.0:1	100.0%
128	50.0%	0.006	1.0:1	100.0%

```
============================================================
ANALYSE DE LA COMPRESSION D'IMAGE PAR SVD
============================================================
Dimensions de l'image : 256 × 256 = 65,536 pixels
Nombre total de valeurs singulières disponibles : 256

Rangs nécessaires pour différents niveaux de qualité :
   90.0% d'énergie → Rang    1 → Compression 127.8:1
   95.0% d'énergie → Rang    2 → Compression 63.9:1
   99.0% d'énergie → Rang   10 → Compression 12.8:1
   99.9% d'énergie → Rang   40 → Compression 3.2:1

La SVD permet une compression efficace en concentrant l'information dans les
premières valeurs singulières.
Pour cette image, on peut obtenir une qualité visuelle acceptable avec␣
 ↪seulement
2 valeurs singulières sur 256 disponibles.
```

Analyse des Visualisations

- **Images compressées** : La première figure présente l'image originale `house.tiff` ainsi que ses approximations à différents rangs. À 5% des valeurs singulières, les détails de l'image, tels que les contours de la maison, sont significativement altérés, mais le taux de compression est élevé. Avec 25% et 50% des valeurs singulières, les détails deviennent plus nets, et à 100%, l'image est identique à l'originale, sans perte d'information.
- **Décroissance des valeurs singulières** : Le premier graphique de la seconde figure montre les valeurs singulières en échelle logarithmique. Une décroissance rapide est observée, indiquant que les premières valeurs singulières concentrent la majorité de l'information de l'image.
- **Énergie cumulée** : Le second graphique illustre l'évolution de l'énergie cumulée en fonction du nombre de valeurs singulières. Des seuils de 90%, 95% et 99% d'énergie conservée sont marqués, révélant qu'un petit pourcentage des valeurs singulières suffit pour capturer la majorité de l'énergie.
- **Erreur relative en fonction du rang** : Le troisième graphique présente l'évolution de l'erreur relative en fonction du rang k sur une échelle log-log. L'erreur diminue significativement à mesure que k augmente, démontrant l'impact des premières valeurs singulières.
- **Compromis erreur-compression** : Le quatrième graphique montre la relation entre l'erreur relative et le taux de compression. À faible taux de compression, l'erreur est plus élevée, mais elle diminue rapidement avec un taux de compression plus élevé, permettant d'identifier un rang k optimal selon les besoins.

- **Distribution des valeurs singulières** : Le cinquième graphique est un histogramme des valeurs singulières en échelle logarithmique. La distribution confirme que la majorité des valeurs singulières sont de faible amplitude, ce qui explique pourquoi une faible portion suffit pour une bonne reconstruction.

5 Questions

Cette section répond aux trois questions posées dans les consignes du projet, en s'appuyant sur les résultats obtenus dans les exercices précédents. L'objectif est de fournir une analyse détaillée des performances des méthodes, de l'impact de la réduction de dimension via la SVD, et des avantages et limites de l'ACP, en intégrant les points mentionnés dans le notebook de manière fluide et enrichie.

5.1 Comparaison des performances (temps, précision) des méthodes directes et itératives pour résoudre $Ax = b$

Les méthodes directes, telles que la décomposition LU et la décomposition de Cholesky, offrent des solutions précises et garantissent la convergence en un nombre fini d'étapes.
Dans notre cas, pour une matrice 4×4, ces méthodes produisent des erreurs résiduelles de l'ordre de 10^{-17}, proches de la précision machine, ce qui reflète leur robustesse pour les petites matrices bien conditionnées. La décomposition LU factorise la matrice A en $PA = LU$, où P est une matrice de permutation pour la stabilité numérique, et résout le système en deux étapes (substitution avant et arrière). La méthode de Cholesky, applicable aux matrices symétriques définies positives, factorise $A = LL^T$, réduisant le coût computationnel d'environ la moitié par rapport à LU grâce à l'exploitation de la symétrie. Dans notre implémentation, les temps d'exécution pour LU et Cholesky sont de l'ordre des microsecondes, mais la différence est minime pour une si petite matrice, où les surcoûts liés aux appels de fonctions dominent.
En revanche, les méthodes itératives, comme Jacobi et Gauss-Seidel, sont conçues pour les matrices creuses ou de grande taille, où les méthodes directes deviennent coûteuses en mémoire et en calculs, avec une complexité de l'ordre de $O(n^3)$ pour les méthodes directes contre $O(n^2)$ par itération pour les méthodes itératives sur des matrices denses. Dans notre exemple, la méthode de Jacobi ne converge pas dans les 100 itérations maximales, produisant une erreur résiduelle massive (10^{27}), probablement parce que la matrice A n'est pas strictement à dominance diagonale, une condition qui favorise la convergence de Jacobi. La méthode de Gauss-Seidel, qui met à jour les composantes séquentiellement en utilisant les valeurs les plus récentes, converge en 37 itérations avec une erreur résiduelle de 10^{-7}, ce qui est acceptable mais bien moins précis que les méthodes directes. Les temps d'exécution des méthodes itératives sont également plus élevés que ceux des méthodes directes pour cette petite matrice. En résumé, pour des matrices de petite taille comme celle utilisée ici (4×4), les méthodes directes sont plus rapides et plus précises, avec des erreurs résiduelles négligeables et des temps d'exécution minimes. Cependant, pour des matrices de grande taille et creuses, comme celles rencontrées dans la simulation physique ou l'analyse de réseaux, les méthodes itératives sont plus efficaces en termes de mémoire, car elles évitent de stocker des

matrices intermédiaires comme L et U. Le choix de la méthode dépend donc du contexte : pour les petits systèmes denses, LU et Cholesky sont préférables ; pour les grands systèmes creux, Jacobi ou Gauss-Seidel (ou des variantes avancées comme le gradient conjugué) sont plus adaptées, à condition que la matrice satisfasse les conditions de convergence.

5.2 Impact de la réduction de dimension via SVD sur l'analyse de données

La décomposition en valeurs singulières (SVD) est un outil puissant pour l'analyse de données, car elle permet de réduire la dimensionnalité tout en préservant l'information essentielle. La SVD factorise une matrice $D \in \mathbb{R}^{m \times n}$ en $D = U\Sigma V^T$, où U et V sont orthogonales, et Σ contient les valeurs singulières décroissantes. En ne conservant que les k plus grandes valeurs singulières, on obtient une approximation de rang k, $D_{\text{approx}} = U[:, : k] \cdot \text{diag}(S[: k]) \cdot V^T[: k, :]$, qui capture les directions de plus grande variance dans les données. Dans notre exemple d'Analyse en Composantes Principales (ACP), la SVD est appliquée à une matrice de données centrée $D_{\text{centrée}}$ de taille 10×5. En projetant les données sur les deux premières composantes principales (lignes de V^T), nous obtenons une représentation 2D qui conserve 89,40 % de la variance totale, comme calculé par $\sum_{i=1}^{2} \sigma_i^2 / \sum_{i=1}^{n} \sigma_i^2$. Cela permet de visualiser des données multidimensionnelles dans un espace de faible dimension, facilitant l'identification des tendances principales. Par exemple, dans un contexte réel, si D représente les notes d'étudiants dans 5 matières, la projection 2D peut révéler des regroupements d'étudiants ayant des profils similaires. La réduction de dimension via SVD a plusieurs impacts bénéfiques :

- **Visualisation** : Elle permet de représenter des données complexes (par exemple, 5 dimensions) dans un espace 2D ou 3D, rendant l'interprétation plus intuitive.
- **Réduction du bruit** : Les petites valeurs singulières sont souvent associées au bruit. En les éliminant, la SVD améliore la qualité des données pour des analyses ultérieures.
- **Compression** : Comme montré dans l'exercice de compression d'image, la SVD réduit la taille des données en stockant uniquement les k premières colonnes de U, les k premières valeurs singulières, et les k premières lignes de V^T, avec un taux de compression de $k \cdot (m + n)/(m \cdot n)$.
- **Efficacité computationnelle** : En réduisant la dimensionnalité, la SVD diminue le coût des calculs dans des algorithmes comme l'apprentissage automatique.

Dans des applications réelles, la SVD est utilisée dans la recommandation (par exemple, systèmes de filtrage collaboratif chez Netflix), l'analyse sémantique latente (LSA) pour le traitement du langage naturel, et la reconnaissance d'images. Cependant, la perte d'information due à l'élimination des petites valeurs singulières peut affecter certaines tâches, comme la classification, où des composantes de faible variance peuvent contenir des informations discriminantes.

5.3 Avantages et limites de l'ACP pour l'analyse de données

L'Analyse en Composantes Principales (ACP), basée sur la SVD, est une méthode fondamentale pour l'analyse de données multidimensionnelles. Elle transforme les données dans une nouvelle base orthogonale où les axes (composantes principales) correspondent aux directions de variance maximale. Dans notre implémentation, l'ACP a permis de réduire une matrice 10×5 à une projection 2D, conservant 89,40 % de la variance, ce qui illustre son efficacité pour capturer les tendances principales. **Avantages de l'ACP :**

- **Réduction de dimension** : L'ACP réduit la complexité des données tout en préservant la majorité de la variance, facilitant les calculs et l'interprétation.
- **Visualisation** : Elle permet de représenter des données multidimensionnelles dans un espace 2D ou 3D, comme montré dans notre graphique de projection.
- **Élimination du bruit** : En éliminant les composantes associées aux petites valeurs singulières, l'ACP réduit l'impact du bruit, améliorant la robustesse des analyses.
- **Applications variées** : L'ACP est utilisée dans la visualisation de données, la compression, et comme étape de prétraitement dans l'apprentissage automatique (par exemple, avant une régression ou une classification).

Limites de l'ACP :

- **Hypothèse de linéarité** : L'ACP suppose que les relations entre variables sont linéaires, ce qui peut ne pas convenir à des données avec des structures non linéaires. Des méthodes comme t-SNE ou UMAP sont mieux adaptées dans ces cas.
- **Sensibilité aux valeurs aberrantes** : Les outliers peuvent fausser les composantes principales, car elles sont basées sur la maximisation de la variance.
- **Perte d'information** : Les composantes de faible variance, éliminées lors de la réduction, peuvent contenir des informations importantes pour certaines tâches, comme la classification.
- **Interprétation complexe** : Les composantes principales sont des combinaisons linéaires des variables originales, ce qui peut rendre leur interprétation difficile, surtout dans des domaines où les variables ont une signification physique (par exemple, température, pression).
- **Structure locale** : L'ACP se concentre sur la variance globale et peut ne pas préserver la structure locale des données, contrairement à des méthodes comme les autoencodeurs non linéaires.

En pratique, l'ACP est particulièrement utile comme étape préliminaire pour explorer les données ou réduire leur dimensionnalité avant d'appliquer des algorithmes plus complexes. Cependant, elle doit souvent être complétée par d'autres techniques si l'objectif est de capturer des relations non linéaires ou de préserver des informations discriminantes pour des tâches spécifiques comme la classification.

6 Résumé et Conclusion

Ce projet a permis d'explorer les concepts fondamentaux de l'algèbre linéaire numérique et leurs applications pratiques à travers des implémentations en Python. La résolution de systèmes linéaires a mis en évidence l'efficacité des méthodes directes (LU, Cholesky) pour les petites matrices denses, et l'adaptabilité des méthodes itératives (Jacobi, Gauss-Seidel) aux matrices creuses ou de grande taille. Les décompositions SVD et QR ont démontré leur utilité pour l'analyse de données, notamment via l'ACP pour la réduction de dimension et la visualisation, et la compression d'image, qui illustre le compromis entre qualité et taille des données. Ces techniques, rigoureusement appliquées, soulignent l'importance de l'algèbre linéaire dans la science des données et le traitement numérique, offrant des outils puissants pour résoudre des problèmes complexes de manière efficace.